Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»

Кафедра вычислительных систем

ОТЧЕТ
по практической работе 7

по дисциплине «**Основы разработки на Java**»

Выполнил:
студент гр. ИС-542
«11» октября 2026 г. _____ /Кузугашев С.В./

Проверил:
Ассистент кафедры ВС
«12» октября 2026 г. _____ /Гузенков С.К./

Оценка «_____»

Новосибирск 2026

Оглавление

Задание 3
Выполнение работы 5
Структура проекта 5
Задача 1. Каталог курсов в UTF-8 5
Задача 2. Buffered Reader/Writer и граница bytes → chars 6
Задача 3. Try-with-resources и порядок закрытия 7
Задача 4. Ленивые строки и владение Stream 8
Задача 5. Бинарный формат каталога 9
Задача 6. Временный файл и atomic move 10
Задача 7. Частично повреждённый импорт 11
Задача 8. Потоковый импорт большого файла 12
Задача 9. Версионируемый формат и миграция 14
Ошибки и их исправление 16
Заключение 17
Приложение 18

Задание

Дисциплина «Программирование». Лабораторная работа №7. Тема: работа с файлами и потоками ввода-вывода Java — кодировки, буферизация, try-with-resources, ленивые Stream, бинарные и версионируемые форматы, атомарная замена и потоковый импорт. Среда: OpenJDK 21.

Исходный код лабораторной работы опубликован в открытом репозитории: https://github.com/Forlein/SibGUTI-Java-Labs (папка WORK_Java/laboratory_work_7).

В работе реализованы 9 заданий:

1. Каталог курсов: сохранение и чтение в UTF-8, обработка отсутствующего файла и каталога вместо файла.

2. Текстовая обработка через BufferedReader/BufferedWriter со счётчиками строк и символов, объяснение границы bytes → chars.

3. Try-with-resources: pipeline с исключением в середине и доказательство правильного порядка close.

4. Files.lines только в try-with-resources; воспроизведение ошибочного возврата Stream наружу.

5. Бинарный формат: magic, version, count и записи через DataOutputStream/DataInputStream; отказ для неизвестной версии и обрезанного файла.

6. Временный файл в том же каталоге, flush/close и замена целевого файла через move с fallback без atomic move.

7. Импорт с ошибками и номерами строк: all-or-nothing, контракт результата зафиксирован sealed-типом.

8. Потоковый импорт большого файла: счётчики, ограниченный список ошибок, отмена после заданного порога.

9. Версионируемый формат: schema version, checksum, миграция v1 → v2, отказ будущей версии.

Работа выполнена на оценку «5»: реализованы все 9 заданий, каждое собрано и запущено, программы выдают корректный результат.

Выполнение работы

Ниже по каждой задаче приведён ключевой фрагмент кода (микро-листинг) и вывод программы; полные исходники лежат в папках 1–9 проекта.

Структура проекта

Каждое задание вынесено в отдельную папку с номером, внутри которой лежит файл Main.java. Классы с одинаковыми именами (Course, Main и т. п.) продублированы в папках, поэтому задания собираются и запускаются по отдельности. Сборка: javac -encoding UTF-8 -d out <папка>\Main.java; запуск: java -Dstdout.encoding=UTF-8 -cp out Main.

Папка	Файлы и задание
1	Main.java — каталог курсов в UTF-8
2	Main.java — BufferedReader/BufferedWriter
3	Main.java — try-with-resources
4	Main.java — Files.lines и владение Stream
5	Main.java — бинарный формат
6	Main.java — временный файл и atomic move
7	Main.java — частично повреждённый импорт
8	Main.java — потоковый импорт большого файла
9	Main.java — версионируемый формат
Корень	README.md, Отчет_ЛР7_Java.docx

Задача 1. Каталог курсов в UTF-8

В первой задаче каталог курсов сохраняется в текстовый файл в кодировке UTF-8 и читается обратно. Перед записью и чтением проверяется, не является ли путь каталогом, а при чтении отдельно обрабатывается отсутствие файла. Ошибка не проглатывается: программа печатает сообщение в stderr и завершается с кодом 1, поэтому ложного успеха не возникает.

Листинг – 1 Сохранение и чтение каталога с проверкой ошибок (1/Main.java)

```
1     try {
2         saveCatalog(catalog, courses);
3         List<Course> loaded = loadCatalog(catalog);
4         for (Course course : loaded) {
```

```
5             System.out.println(course.name() + " — " + course.hours() +
      " ч.");
6         }
7         System.out.println("Каталог сохранён и прочитан: " +
      loaded.size() + " курсов");
8     } catch (IOException | IllegalArgumentException e) {
9         System.err.println("Ошибка каталога курсов: " + e.getMessage());
10        System.exit(1);
11    }
12
13    if (Files.isDirectory(path)) {
14        throw new IOException("по пути " + path.toAbsolutePath()
15                + " находится каталог, а не файл");
16    }
17    if (!Files.exists(path)) {
18        throw new IOException("файл не найден: " +
      path.toAbsolutePath());
19    }
```

Вывод программы (1/Main.java)

```
1     Java. Базовый курс — 72 ч.
2     Алгоритмы и структуры данных — 108 ч.
3     Базы данных — 64 ч.
4     Веб-разработка — 96 ч.
5     Каталог сохранён и прочитан: 4 курсов
```

Программа сохраняет четыре курса в courses.txt в UTF-8, затем читает файл и печатает записи. Отсутствие файла и путь-каталог не игнорируются: выбрасывается IOException, сообщение выводится в stderr, процесс завершается с кодом 1.

Задача 2. Buffered Reader/Writer и граница bytes → chars

Во второй задаче текстовая обработка переписана через BufferedReader и BufferedWriter. InputStreamReader декодирует байты файла в UTF-16 char, а OutputStreamWriter кодирует char обратно в UTF-8 bytes; буферы только уменьшают число обращений к файлу. Программа считает строки, UTF-16 char и Unicode code points, поэтому видно, что эмодзи занимает две char.

Листинг – 2 Буферизованное чтение, запись и счётчики (2/Main.java)

```
1     try (BufferedReader reader = new BufferedReader(
2             new InputStreamReader(Files.newInputStream(input),
      StandardCharsets.UTF_8));
```

```
3          BufferedWriter writer = new BufferedWriter(
4                  new OutputStreamWriter(Files.newOutputStream(output),
    StandardCharsets.UTF_8))) {
5          String line;
6          while ((line = reader.readLine()) != null) {
7              lines++;
8              chars += line.length();
9              codePoints += line.codePointCount(0, line.length());
10             writer.write(lines + ": " + line);
11             writer.newLine();
12         }
13         writer.flush();
14     }
```

Вывод программы (2/Main.java)

```
1    Создан демонстрационный файл: input.txt
2    Строк: 3, UTF-16 char: 43, code points: 42
3    Граница bytes -> chars: InputStream отдаёт байты, InputStreamReader
     декодирует их в UTF-16 char; BufferedWriter кодирует char обратно в
     UTF-8 bytes.
```

BufferedReader.readLine() возвращает уже декодированные String, поэтому счётчик строк равен 3. UTF-16 char — 43, code points — 42: эмодзи 😀 занимает суррогатную пару. Граница bytes → chars находится в InputStreamReader, а обратная граница chars → bytes — в OutputStreamWriter.

Задача 3. Try-with-resources и порядок закрытия

В третьей задаче собран pipeline чтения и записи, внутри которого после flush искусственно выбрасывается IOException. Оба ресурса объявлены в try-with-resources, поэтому закрываются автоматически. Ресурсы обёрнуты в TrackedReader и TrackedWriter, которые записывают порядок close; дополнительно проверяются флаги закрытия.

Листинг – 3 Pipeline с исключением и трассировкой close (3/Main.java)

```
1    try (TrackedReader reader = new TrackedReader(
2                Files.newBufferedReader(input,
     StandardCharsets.UTF_8));
3         TrackedWriter writer = new TrackedWriter(
4                Files.newBufferedWriter(output,
     StandardCharsets.UTF_8))) {
5        String line = reader.readLine();
6        writer.write(line == null ? "" : line.toUpperCase(Locale.ROOT));
```

```
7         writer.newLine();
8         writer.flush();
9         throw new IOException("искусственный сбой в середине pipeline");
10    }
11
12    @Override
13    public void close() throws IOException {
14        writerClosed = true;
15        CLOSE_ORDER.add("writer.close");
16        delegate.close();
17    }
```

Вывод программы (3/Main.java)

```
1     Перехвачено: искусственный сбой в середине pipeline
2     Порядок close: [writer.close, reader.close]
3     reader закрыт: true, writer закрыт: true
4     Оба ресурса закрыты; порядок обратен порядку объявления.
```

Искусственное исключение возникает после flush, но до выхода из try. try-with-resources закрывает writer, затем reader; в логе видно writer.close → reader.close. Флаги подтверждают, что оба ресурса закрыты, а порядок обратен порядку объявления.

Задача 4. Ленивые строки и владение Stream

В четвёртой задаче показано, что Files.lines возвращает ленивый Stream, который владеет файловым дескриптором. Ошибочный метод возвращает Stream наружу: terminal operation count() не закрывает поток, и onClose остаётся невызванным. Корректный метод потребляет Stream внутри try-with-resources, поэтому дескриптор закрывается автоматически.

Листинг – 4 Ошибочный возврат Stream и корректный try-with-resources (4/Main.java)

```
1     private static Stream<String> brokenLines(Path path) throws
      IOException {
2         return Files.lines(path, StandardCharsets.UTF_8);
3     }
4
5     private static long countLines(Path path) throws IOException {
6         try (Stream<String> lines = Files.lines(path,
      StandardCharsets.UTF_8)) {
7             return lines.count();
```

```
8         }
9     }
10
11    AtomicBoolean closed = new AtomicBoolean(false);
12    Stream<String> leaked = brokenLines(brokenPath).onClose(() ->
      closed.set(true));
13    System.out.println("Прочитано строк: " + leaked.count());
14    System.out.println("onClose вызван после terminal operation: " +
      closed.get());
15    leaked.close();
```

Вывод программы (4/Main.java)

```
1     --- Ошибочный вариант: stream возвращён наружу ---
2     Прочитано строк: 3
3     onClose вызван после terminal operation: false
4     Удаление при открытом stream: удалён
5     onClose после leaked.close(): true
6     После leaked.close(): файл уже отсутствует
7     Владелец ресурса: тот, кто открыл Files.lines(); при возврате stream
      наружу закрывать обязан вызывающий.
8     --- Корректный вариант: try-with-resources ---
9     Корректно посчитано строк: 2
10    После корректного try-with-resources: удалён
```

После terminal operation count() onClose не вызван — поток остаётся открытым, хотя ОС разрешила удалить файл. Только leaked.close() переводит флаг в true. В корректном варианте Files.lines находится в try-with-resources, поэтому дескриптор закрывается автоматически.

Задача 5. Бинарный формат каталога

В пятой задаче каталог записывается в бинарный файл через DataOutputStream: magic header, version, количество записей и сами записи (writeUTF для названия и writeInt для часов). Чтение выполняется через DataInputStream. Неизвестная версия отклоняется до чтения записей, а обрезанный файл даёт EOFException, который преобразуется в понятное исключение.

Листинг – 5 Заголовок и записи бинарного формата (5/Main.java)

```
1     try (DataOutputStream out = new DataOutputStream(
2             new BufferedOutputStream(Files.newOutputStream(path)))) {
3         out.writeInt(MAGIC);
```

```
4           out.writeInt(VERSION);
5           out.writeInt(courses.size());
6           for (Course course : courses) {
7               out.writeUTF(course.name());
8               out.writeInt(course.hours());
9           }
10      }
11
12      int magic = in.readInt();
13      if (magic != MAGIC) {
14          throw new InvalidCatalogException("неверный magic header");
15      }
16      int version = in.readInt();
17      if (version != VERSION) {
18          throw new InvalidCatalogException("неизвестная версия: " +
        version);
19      }
20      int count = in.readInt();
21      if (count < 0 || count > MAX_RECORDS) {
22          throw new InvalidCatalogException("недопустимое количество
        записей: " + count);
23      }
```

Вывод программы (5/Main.java)

```
1       Прочитан каталог: [Course[name=Java. Базовый курс, hours=72],
        Course[name=Алгоритмы и структуры данных, hours=108],
        Course[name=Базы данных, hours=64]]
2       неизвестная версия отклонён: неизвестная версия: 99
3       обрезанный файл отклонён: файл обрезан
```

Валидный файл читается: magic и version совпали, count=3. Файл с version=99 отклонён до чтения записей; обрезанный файл даёт EOFException, который преобразован в InvalidCatalogException.

Задача 6. Временный файл и atomic move

В шестой задаче новый каталог сначала записывается во временный файл, созданный в том же каталоге, что и целевой. После flush и close выполняется move с REPLACE_EXISTING и ATOMIC_MOVE. Если файловая система не поддерживает atomic move, программа выполняет обычную замену и явно сообщает, что атомарность не гарантирована.

Листинг – 6 Временный файл и замена целевого файла (6/Main.java)

```
    Path temp = Files.createTempFile(directory, target.getFileName() +
    ".", ".tmp");
    boolean moved = false;
    try {
        try (BufferedWriter writer = Files.newBufferedWriter(temp,
    StandardCharsets.UTF_8)) {
            for (String line : lines) {
                writer.write(line);
                writer.newLine();
            }
            writer.flush();
        }
        moveReplacing(temp, target);
        moved = true;
    } finally {
        if (!moved) {
            Files.deleteIfExists(temp);
        }
    }

    try {
        Files.move(temp, target, StandardCopyOption.REPLACE_EXISTING,
                StandardCopyOption.ATOMIC_MOVE);
    } catch (AtomicMoveNotSupportedException e) {
        Files.move(temp, target, StandardCopyOption.REPLACE_EXISTING);
    }
```

Вывод программы (6/Main.java)

```
Замена выполнена atomic move.
Содержимое после замены:
Java. Базовый курс
Базы данных
```

Новый каталог записан во временный файл в том же каталоге, затем заменяет target через ATOMIC_MOVE. На NTFS atomic move поддерживается; при AtomicMoveNotSupportedException выполняется обычный move с предупреждением.

Задача 7. Частично повреждённый импорт

В седьмой задаче импортёр собирает ошибки с номерами строк и не публикует половину каталога. Выбран контракт all-or-nothing: результат импорта представлен sealed-типом ImportResult с вариантами ImportSuccess и

ImportFailure. При любой ошибке возвращается ImportFailure со списком ImportError, а хранилище не получает частично разобранный каталог.

Листинг – 7 Контракт результата импорта (7/Main.java)

```
1     private sealed interface ImportResult permits ImportSuccess,
      ImportFailure {
2     }
3
4     private record ImportSuccess(List<Course> courses) implements
      ImportResult {
5     }
6
7     private record ImportFailure(List<ImportError> errors) implements
      ImportResult {
8     }
9
10    private record ImportError(int lineNumber, String message) {
11    }
12
13    if (!errors.isEmpty()) {
14        return new ImportFailure(List.copyOf(errors));
15    }
16    return new ImportSuccess(List.copyOf(parsed));
```

Вывод программы (7/Main.java)

```
1     До импорта: [Course[name=Старый курс, hours=10]]
2     Импорт отклонён, ошибок: 2
3       строка 2: неверное число часов: сто
4       строка 3: пустое название
5     После отклонённого импорта: [Course[name=Старый курс, hours=10]]
6     Успешный импорт опубликован: [Course[name=Java, hours=72],
      Course[name=Базы данных, hours=64]]
```

Повреждённый импорт вернул ImportFailure с ошибками в строках 2 и 3. store не получил частичный каталог: опубликованный список остался прежним. Корректный файл вернул ImportSuccess и был опубликован целиком.

Задача 8. Потоковый импорт большого файла

В восьмой задаче большой файл читается построчно через BufferedReader, без readAllLines. Счётчики строк, валидных записей и ошибок имеют постоянный размер, а список ошибок ограничен. После достижения

заданного порога ошибок импорт отменяется, поэтому память не растёт вместе с размером файла.

Листинг – 8 Построчный импорт с ограниченными ошибками (8/Main.java)

```
1     try (BufferedReader reader = Files.newBufferedReader(path,
      StandardCharsets.UTF_8)) {
2         String line;
3         while ((line = reader.readLine()) != null) {
4             totalLines++;
5             String[] parts = line.split(";", -1);
6             if (parts.length != 2) {
7                 collector.add(new ImportError(totalLines, "ожидается 2
      поля"));
8             }
9             if (collector.totalErrors() >= cancelAfterErrors) {
10                cancelled = true;
11                break;
12            }
13        }
14    }
15    return new ImportSummary(totalLines, validRecords,
      collector.totalErrors(),
16            collector.storedErrors(), cancelled);
17
18    private void add(ImportError error) {
19        total++;
20        if (stored.size() < maxStored) {
21            stored.add(error);
22        }
23    }
```

Вывод программы (8/Main.java)

```
1     --- Небольшой файл: ошибок меньше порога ---
2     Обработано строк: 5000
3     Корректных записей: 4995
4     Всего ошибок: 5
5     Сохранено ошибок: 5
6       строка 1000: неверное число часов: ошибка
7       строка 2000: неверное число часов: ошибка
8       строка 3000: неверное число часов: ошибка
9       строка 4000: неверное число часов: ошибка
10      строка 5000: неверное число часов: ошибка
11    Импорт отменён: false
12    --- Большой файл: отмена по порогу ---
13    Обработано строк: 10000
```

```
Корректных записей: 9990
Всего ошибок: 10
Сохранено ошибок: 5
  строка 1000: неверное число часов: ошибка
  строка 2000: неверное число часов: ошибка
  строка 3000: неверное число часов: ошибка
  строка 4000: неверное число часов: ошибка
  строка 5000: неверное число часов: ошибка
Импорт отменён: true
Память: файл читается построчно, хранится не более 5 ошибок, валидные записи не накапливаются.
```

Малый файл обработан полностью: 5000 строк, 5 ошибок, все 5 сохранены. На большом файле после 10-й ошибки импорт остановлен: обработано 10000 строк, сохранено только 5 ошибок, валидные записи не накапливались.

Задача 9. Версионируемый формат и миграция

В девятой задаче формат получил schema version и checksum. Заголовок содержит magic, версию, длину payload и CRC32; checksum проверяется до разбора объектов, поэтому повреждение обнаруживается до публикации. Версия 1 мигрирует в версию 2 с вычислением credits, а неизвестная будущая версия не читается как старая.

Листинг – 9 Версия, checksum и миграция v1 → v2 (9/Main.java)

```
version = in.readInt();
if (version != VERSION_1 && version != VERSION_2) {
    throw new UnsupportedVersionException(version);
}
payloadLength = in.readInt();
expectedChecksum = in.readInt();
byte[] payload = in.readNBytes(payloadLength);
CRC32 crc = new CRC32();
crc.update(payload);
if ((int) crc.getValue() != expectedChecksum) {
    throw new CatalogFormatException("checksum не совпал: данные повреждены");
}
return parsePayload(version, payload);

int credits = version == VERSION_1 ? creditsFromHours(hours) : in.readInt();
```

Вывод программы (9/Main.java)

```
До публикации: [Course[name=Старый курс, hours=36, credits=1]]
v2 прочитан (версия 2): [Course[name=Java. Базовый курс, hours=72, credits=2], Course[name=Базы данных, hours=108, credits=3]]
v1 миграция -> v2 (версия 1): [Course[name=Java. Базовый курс, hours=72, credits=2], Course[name=Базы данных, hours=108, credits=3]]
будущая версия отклонён: неизвестная версия формата: 3
повреждённый файл отклонён: checksum не совпал: данные повреждены
Опубликованный каталог не изменён после повреждения.
```

v2 читается напрямую; v1 мигрирует в v2 с вычислением credits. Версия 3 отклонена до разбора payload. Повреждение последнего байта обнаружено по CRC32 до публикации, поэтому опубликованный каталог не изменился.

Ошибки и их исправление

Отсутствие файла и каталог вместо файла (задача 1). Files.exists() и Files.isDirectory() проверяются до чтения и записи; вместо молчаливого пропуска выбрасывается IOException, и программа завершается с ненулевым кодом.

Кодировка UTF-8 (задачи 1–2). Кодировка платформы может отличаться, поэтому во всех текстовых операциях явно указан StandardCharsets.UTF_8, а сборка и запуск выполняются с -encoding UTF-8 и -Dstdout.encoding=UTF-8.

Порядок закрытия ресурсов (задача 3). try-with-resources закрывает ресурсы в обратном порядке объявления; для pipeline это writer, затем reader. Ручной close в неправильном порядке мог бы потерять буферизованные данные.

Утечка файлового дескриптора (задача 4). Files.lines возвращает ленивый Stream, владеющий дескриптором. Возврат наружу без закрытия оставляет ресурс открытым; корректно потреблять поток внутри try-with-resources.

Неизвестная версия и обрезанный файл (задачи 5, 9). Версия проверяется до разбора записей, а EOF и несовпадение CRC32 преобразуются в понятные исключения; частично прочитанные объекты не публикуются.

Атомарная замена и частичный импорт (задачи 6–7). Временный файл создаётся в том же каталоге, чтобы move был атомарным; при отсутствии ATOMIC_MOVE выполняется fallback с предупреждением. Импорт all-or-nothing не публикует половину каталога.

Заключение

В ходе выполнения лабораторной работы №7 были изучены и применены на практике средства файлового ввода-вывода Java (OpenJDK 21): кодировки, буферизация, try-with-resources, ленивые Stream, бинарные и версионируемые форматы, атомарная замена и потоковая обработка больших файлов.

В задачах 1–3 отработаны текстовая обработка и управление ресурсами: явная граница bytes → chars через InputStreamReader/OutputStreamWriter, счётчики строк и символов, а также гарантированное закрытие reader и writer в обратном порядке.

В задачах 4–6 исследованы ленивые потоки и надёжная публикация файлов: доказано, что Files.lines требует try-with-resources, реализованы бинарный формат с magic/version/count и замена целевого файла через временный файл и atomic move.

В задачах 7–9 построены контракты импорта и версионирования: all-or-nothing результат с номерами строк, потоковый импорт с ограниченной памятью и отменой по порогу, schema version с CRC32 и миграцией v1 → v2.

Все 9 заданий размещены в отдельных папках, каждое задание компилируется и выполняется без ошибок. Работа выполнена в полном объёме.

Приложение

Листинг – 10 1/Main.java

```
1     import java.io.IOException;
2     import java.nio.charset.StandardCharsets;
3     import java.nio.file.Files;
4     import java.nio.file.Path;
5     import java.nio.file.Paths;
6     import java.nio.file.StandardOpenOption;
7     import java.util.ArrayList;
8     import java.util.List;
9
10    public class Main {
11        private static final Path DEFAULT_CATALOG =
      Paths.get("courses.txt");
12
13        public static void main(String[] args) {
14            Path catalog = args.length > 0 ? Paths.get(args[0]) :
      DEFAULT_CATALOG;
15            List<Course> courses = List.of(
16                    new Course("Java. Базовый курс", 72),
17                    new Course("Алгоритмы и структуры данных", 108),
18                    new Course("Базы данных", 64),
19                    new Course("Веб-разработка", 96)
20            );
21
22            try {
23                saveCatalog(catalog, courses);
24                List<Course> loaded = loadCatalog(catalog);
25                for (Course course : loaded) {
26                    System.out.println(course.name() + " — " +
      course.hours() + " ч.");
27                }
28                System.out.println("Каталог сохранён и прочитан: " +
      loaded.size() + " курсов");
29            } catch (IOException | IllegalArgumentException e) {
30                System.err.println("Ошибка каталога курсов: " +
      e.getMessage());
31                System.exit(1);
32            }
33        }
34
35        private static void saveCatalog(Path path, List<Course> courses)
      throws IOException {
36            requireNotDirectory(path);
37            List<String> lines = new ArrayList<>();
```

```
38          for (Course course : courses) {
39              lines.add(course.name() + "\t" + course.hours());
40          }
41          Files.write(path, lines, StandardCharsets.UTF_8,
42                  StandardOpenOption.CREATE,
    StandardOpenOption.TRUNCATE_EXISTING, StandardOpenOption.WRITE);
43      }
44
45      private static List<Course> loadCatalog(Path path) throws
    IOException {
46          if (Files.isDirectory(path)) {
47              throw new IOException("по пути " + path.toAbsolutePath()
    + " находится каталог, а не файл");
48          }
49          if (!Files.exists(path)) {
50              throw new IOException("файл не найден: " +
    path.toAbsolutePath());
51          }
52          List<String> lines = Files.readAllLines(path,
    StandardCharsets.UTF_8);
53          List<Course> courses = new ArrayList<>();
54          for (int i = 0; i < lines.size(); i++) {
55              String line = lines.get(i);
56              if (line.isBlank()) {
57                  continue;
58              }
59              String[] parts = line.split("\t", -1);
60              if (parts.length != 2) {
61                  throw new IOException("строка " + (i + 1) + " имеет
    неверный формат");
62              }
63              try {
64                  courses.add(new Course(parts[0],
    Integer.parseInt(parts[1])));
65              } catch (NumberFormatException e) {
66                  throw new IOException("строка " + (i + 1) + "
    содержит неверное число часов", e);
67              }
68          }
69          return courses;
70      }
71
72      private static void requireNotDirectory(Path path) throws
    IOException {
73          if (Files.isDirectory(path)) {
74              throw new IOException("по пути " + path.toAbsolutePath()
    + " находится каталог, а не файл");
75          }
76      }
```

```
77
78        private record Course(String name, int hours) {
79            private Course {
80                if (name == null || name.isBlank()) {
81                    throw new IllegalArgumentException("название курса
    не должно быть пустым");
82                }
83                if (hours <= 0) {
84                    throw new IllegalArgumentException("часы курса
    должны быть положительными");
85                }
86            }
87        }
88    }
```

Листинг – 11 2/Main.java

```
1     import java.io.BufferedReader;
2     import java.io.BufferedWriter;
3     import java.io.IOException;
4     import java.io.InputStreamReader;
5     import java.io.OutputStreamWriter;
6     import java.nio.charset.StandardCharsets;
7     import java.nio.file.Files;
8     import java.nio.file.Path;
9     import java.nio.file.Paths;
10    import java.util.Locale;
11
12    public class Main {
13        public static void main(String[] args) {
14            Path input = args.length > 0 ? Paths.get(args[0]) :
    Paths.get("input.txt");
15            Path output = args.length > 1 ? Paths.get(args[1]) :
    Paths.get("output.txt");
16
17            try {
18                if (!Files.exists(input)) {
19                    Files.writeString(input, "Привет, мир!\nJava и
    Unicode: 😀\nТретья строка\n", StandardCharsets.UTF_8);
20                    System.out.println("Создан демонстрационный файл: "
    + input.toAbsolutePath());
21                }
22
23                Statistics statistics = process(input, output);
24                System.out.printf(Locale.ROOT, "Строк: %d, UTF-16 char:
    %d, code points: %d%n",
25                        statistics.lines(), statistics.chars(),
    statistics.codePoints());
```

```
26              System.out.println("Граница bytes -> chars: InputStream
    отдаёт байты, InputStreamReader декодирует их в UTF-16 char;
    BufferedWriter кодирует char обратно в UTF-8 bytes.");
27          } catch (IOException e) {
28              System.err.println("Ошибка обработки текста: " +
    e.getMessage());
29              System.exit(1);
30          }
31      }
32
33      private static Statistics process(Path input, Path output)
    throws IOException {
34          if
    (input.toAbsolutePath().normalize().equals(output.toAbsolutePath().n
    ormalize())) {
35              throw new IOException("входной и выходной файлы должны
    различаться");
36          }
37
38          long lines = 0;
39          long chars = 0;
40          long codePoints = 0;
41
42          try (BufferedReader reader = new BufferedReader(
43                  new InputStreamReader(Files.newInputStream(input),
    StandardCharsets.UTF_8));
44               BufferedWriter writer = new BufferedWriter(
45                       new
    OutputStreamWriter(Files.newOutputStream(output),
    StandardCharsets.UTF_8))) {
46
47              String line;
48              while ((line = reader.readLine()) != null) {
49                  lines++;
50                  chars += line.length();
51                  codePoints += line.codePointCount(0, line.length());
52                  writer.write(lines + ": " + line);
53                  writer.newLine();
54              }
55              writer.flush();
56          }
57
58          return new Statistics(lines, chars, codePoints);
59      }
60
61      private record Statistics(long lines, long chars, long
    codePoints) {
62      }
```

```
63    }
```

Листинг – 12 3/Main.java

```
1     import java.io.BufferedReader;
2     import java.io.BufferedWriter;
3     import java.io.Closeable;
4     import java.io.IOException;
5     import java.nio.charset.StandardCharsets;
6     import java.nio.file.Files;
7     import java.nio.file.Path;
8     import java.util.ArrayList;
9     import java.util.List;
10    import java.util.Locale;
11
12    public class Main {
13        private static final List<String> CLOSE_ORDER = new
      ArrayList<>();
14        private static boolean readerClosed;
15        private static boolean writerClosed;
16
17        public static void main(String[] args) {
18            boolean ok;
19            try {
20                ok = demonstrate();
21            } catch (IOException e) {
22                System.err.println("Ошибка демонстрации: " +
      e.getMessage());
23                ok = false;
24            }
25            if (!ok) {
26                System.exit(1);
27            }
28        }
29
30        private static boolean demonstrate() throws IOException {
31            Path dir = Files.createTempDirectory("try-with-resources-");
32            Path input = dir.resolve("input.txt");
33            Path output = dir.resolve("output.txt");
34
35            try {
36                Files.writeString(input, "данные для pipeline\n",
      StandardCharsets.UTF_8);
37
38                try {
39                    runPipeline(input, output);
```

```
40                  System.out.println("ОШИБКА: исключение не
    возникло");
41                  return false;
42              } catch (IOException e) {
43                  System.out.println("Перехвачено: " +
    e.getMessage());
44              }
45
46              System.out.println("Порядок close: " + CLOSE_ORDER);
47              System.out.println("reader закрыт: " + readerClosed + ",
    writer закрыт: " + writerClosed);
48
49              boolean correctOrder =
    CLOSE_ORDER.equals(List.of("writer.close", "reader.close"));
50              if (!readerClosed || !writerClosed || !correctOrder) {
51                  System.err.println("ОШИБКА: ресурсы закрыты
    неверно");
52                  return false;
53              }
54
55              System.out.println("Оба ресурса закрыты; порядок обратен
    порядку объявления.");
56              return true;
57          } finally {
58              Files.deleteIfExists(output);
59              Files.deleteIfExists(input);
60              Files.deleteIfExists(dir);
61          }
62      }
63
64      private static void runPipeline(Path input, Path output) throws
    IOException {
65          try (TrackedReader reader = new
    TrackedReader(Files.newBufferedReader(input,
    StandardCharsets.UTF_8));
66               TrackedWriter writer = new
    TrackedWriter(Files.newBufferedWriter(output,
    StandardCharsets.UTF_8))) {
67
68              String line = reader.readLine();
69              writer.write(line == null ? "" :
    line.toUpperCase(Locale.ROOT));
70              writer.newLine();
71              writer.flush();
72
73              throw new IOException("искусственный сбой в середине
    pipeline");
74          }
```

```
75        }
76
77        private static final class TrackedReader implements Closeable {
78            private final BufferedReader delegate;
79
80            private TrackedReader(BufferedReader delegate) {
81                this.delegate = delegate;
82            }
83
84            private String readLine() throws IOException {
85                return delegate.readLine();
86            }
87
88            @Override
89            public void close() throws IOException {
90                readerClosed = true;
91                CLOSE_ORDER.add("reader.close");
92                delegate.close();
93            }
94        }
95
96        private static final class TrackedWriter implements Closeable {
97            private final BufferedWriter delegate;
98
99            private TrackedWriter(BufferedWriter delegate) {
100               this.delegate = delegate;
101           }
102
103           private void write(String text) throws IOException {
104               delegate.write(text);
105           }
106
107           private void newLine() throws IOException {
108               delegate.newLine();
109           }
110
111           private void flush() throws IOException {
112               delegate.flush();
113           }
114
115           @Override
116           public void close() throws IOException {
117               writerClosed = true;
118               CLOSE_ORDER.add("writer.close");
```

```
119             delegate.close();
120         }
121     }
122 }
```

Листинг – 13 4/Main.java

```
1   import java.io.IOException;
2   import java.nio.charset.StandardCharsets;
3   import java.nio.file.Files;
4   import java.nio.file.Path;
5   import java.util.concurrent.atomic.AtomicBoolean;
6   import java.util.stream.Stream;
7
8   public class Main {
9       public static void main(String[] args) {
10          Path brokenPath = null;
11          Path correctPath = null;
12          boolean ok = true;
13
14          try {
15              brokenPath = Files.createTempFile("lazy-lines-broken-",
    ".txt");
16              Files.writeString(brokenPath, "первая строка\nвторая
    строка\nтретья строка\n", StandardCharsets.UTF_8);
17
18              System.out.println("--- Ошибочный вариант: stream
    возвращён наружу ---");
19              AtomicBoolean closed = new AtomicBoolean(false);
20              Stream<String> leaked =
    brokenLines(brokenPath).onClose(() -> closed.set(true));
21              System.out.println("Прочитано строк: " +
    leaked.count());
22              System.out.println("onClose вызван после terminal
    operation: " + closed.get());
23              System.out.println("Удаление при открытом stream: " +
    deleteStatus(brokenPath));
24              leaked.close();
25              System.out.println("onClose после leaked.close(): " +
    closed.get());
26              System.out.println("После leaked.close(): " +
    deleteStatus(brokenPath));
27              System.out.println("Владелец ресурса: тот, кто открыл
    Files.lines(); при возврате stream наружу закрывать обязан
    вызывающий.");
28
29              correctPath = Files.createTempFile("lazy-lines-correct-
    ", ".txt");
```

```
30              Files.writeString(correctPath, "альфа\nбета\n",
    StandardCharsets.UTF_8);
31
32              System.out.println("--- Корректный вариант: try-with-
    resources ---");
33              long count = countLines(correctPath);
34              System.out.println("Корректно посчитано строк: " +
    count);
35              String status = deleteStatus(correctPath);
36              System.out.println("После корректного try-with-
    resources: " + status);
37              if (!status.equals("удалён")) {
38                  ok = false;
39              }
40          } catch (IOException e) {
41              System.err.println("Ошибка: " + e.getMessage());
42              ok = false;
43          } finally {
44              deleteQuietly(brokenPath);
45              deleteQuietly(correctPath);
46          }
47
48          if (!ok) {
49              System.exit(1);
50          }
51      }
52
53      private static Stream<String> brokenLines(Path path) throws
    IOException {
54          return Files.lines(path, StandardCharsets.UTF_8);
55      }
56
57      private static long countLines(Path path) throws IOException {
58          try (Stream<String> lines = Files.lines(path,
    StandardCharsets.UTF_8)) {
59              return lines.count();
60          }
61      }
62
63      private static String deleteStatus(Path path) {
64          try {
65              return Files.deleteIfExists(path) ? "удалён" : "файл уже
    отсутствует";
66          } catch (IOException e) {
67              return "не удалось (" + e.getClass().getSimpleName() +
    ") — дескриптор ещё открыт";
68          }
```

```
69        }
70
71        private static void deleteQuietly(Path path) {
72            if (path == null) {
73                return;
74            }
75            try {
76                Files.deleteIfExists(path);
77            } catch (IOException ignored) {
78            }
79        }
80    }
```

Листинг – 14 5/Main.java

```
1     import java.io.BufferedInputStream;
2     import java.io.BufferedOutputStream;
3     import java.io.DataInputStream;
4     import java.io.DataOutputStream;
5     import java.io.EOFException;
6     import java.io.IOException;
7     import java.io.RandomAccessFile;
8     import java.io.UTFDataFormatException;
9     import java.nio.file.Files;
10    import java.nio.file.Path;
11    import java.util.ArrayList;
12    import java.util.Comparator;
13    import java.util.List;
14    import java.util.stream.Stream;
15
16    public class Main {
17        private static final int MAGIC = 0x43415431;
18        private static final int VERSION = 1;
19        private static final int MAX_RECORDS = 10_000;
20
21        public static void main(String[] args) {
22            Path dir = null;
23            boolean ok = true;
24
25            try {
26                dir = Files.createTempDirectory("binary-catalog-");
27                Path valid = dir.resolve("valid.bin");
28                Path badVersion = dir.resolve("bad-version.bin");
29                Path truncated = dir.resolve("truncated.bin");
30
31                List<Course> courses = List.of(
32                        new Course("Java. Базовый курс", 72),
```

```
33                      new Course("Алгоритмы и структуры данных", 108),
34                      new Course("Базы данных", 64)
35              );
36
37              writeCatalog(valid, courses);
38              List<Course> loaded = readCatalog(valid);
39              System.out.println("Прочитан каталог: " + loaded);
40
41              writeRawHeader(badVersion, MAGIC, 99, 0);
42              ok &= expectRejected(badVersion, "неизвестная версия");
43
44              writeCatalog(truncated, courses);
45              try (RandomAccessFile file = new
    RandomAccessFile(truncated.toFile(), "rw")) {
46                  file.setLength(Math.max(0, file.length() - 2));
47              }
48              ok &= expectRejected(truncated, "обрезанный файл");
49          } catch (IOException e) {
50              System.err.println("Ошибка бинарного формата: " +
    e.getMessage());
51              ok = false;
52          } finally {
53              deleteRecursively(dir);
54          }
55
56          if (!ok) {
57              System.exit(1);
58          }
59      }
60
61      private static void writeCatalog(Path path, List<Course>
    courses) throws IOException {
62          try (DataOutputStream out = new DataOutputStream(
63                  new
    BufferedOutputStream(Files.newOutputStream(path)))) {
64              out.writeInt(MAGIC);
65              out.writeInt(VERSION);
66              out.writeInt(courses.size());
67              for (Course course : courses) {
68                  out.writeUTF(course.name());
69                  out.writeInt(course.hours());
70              }
71          }
72      }
73
74      private static List<Course> readCatalog(Path path) throws
    IOException {
```

```
75          try (DataInputStream in = new DataInputStream(
76                  new
    BufferedInputStream(Files.newInputStream(path)))) {
77              try {
78                  int magic = in.readInt();
79                  if (magic != MAGIC) {
80                      throw new InvalidCatalogException("неверный
    magic header");
81                  }
82
83                  int version = in.readInt();
84                  if (version != VERSION) {
85                      throw new InvalidCatalogException("неизвестная
    версия: " + version);
86                  }
87
88                  int count = in.readInt();
89                  if (count < 0 || count > MAX_RECORDS) {
90                      throw new InvalidCatalogException("недопустимое
    количество записей: " + count);
91                  }
92
93                  List<Course> courses = new ArrayList<>(count);
94                  for (int i = 0; i < count; i++) {
95                      String name = in.readUTF();
96                      int hours = in.readInt();
97                      if (name.isBlank()) {
98                          throw new InvalidCatalogException("пустое
    название в записи #" + (i + 1));
99                      }
100                     if (hours <= 0) {
101                         throw new InvalidCatalogException("неверные
    часы в записи #" + (i + 1));
102                     }
103                     courses.add(new Course(name, hours));
104                 }
105
106                 if (in.read() != -1) {
107                     throw new InvalidCatalogException("лишние данные
    после записей");
108                 }
109                 return courses;
110             } catch (EOFException e) {
111                 throw new InvalidCatalogException("файл обрезан",
    e);
112             } catch (UTFDataFormatException e) {
113                 throw new InvalidCatalogException("повреждённая
    строка в файле", e);
```

```
114             }
115         }
116     }
117
118     private static void writeRawHeader(Path path, int magic, int
    version, int count) throws IOException {
119         try (DataOutputStream out = new DataOutputStream(
120                 new
    BufferedOutputStream(Files.newOutputStream(path)))) {
121             out.writeInt(magic);
122             out.writeInt(version);
123             out.writeInt(count);
124         }
125     }
126
127     private static boolean expectRejected(Path path, String label) {
128         try {
129             List<Course> courses = readCatalog(path);
130             System.out.println("ОШИБКА: " + label + " был принят: "
    + courses);
131             return false;
132         } catch (InvalidCatalogException e) {
133             System.out.println(label + " отклонён: " +
    e.getMessage());
134             return true;
135         } catch (IOException e) {
136             System.out.println(label + " отклонён (I/O): " +
    e.getMessage());
137             return true;
138         }
139     }
140
141     private static void deleteRecursively(Path dir) {
142         if (dir == null) {
143             return;
144         }
145         try (Stream<Path> paths = Files.walk(dir)) {
146             paths.sorted(Comparator.reverseOrder()).forEach(path ->
    {
147                 try {
148                     Files.deleteIfExists(path);
149                 } catch (IOException ignored) {
150                 }
151             });
152         } catch (IOException ignored) {
153         }
154     }
```

```
155
156       private record Course(String name, int hours) {
157       }
158
159       private static final class InvalidCatalogException extends
      IOException {
160           private static final long serialVersionUID = 1L;
161
162           private InvalidCatalogException(String message) {
163               super(message);
164           }
165
166           private InvalidCatalogException(String message, Throwable
      cause) {
167               super(message, cause);
168           }
169       }
170   }
```

Листинг – 15 6/Main.java

```
1     import java.io.BufferedWriter;
2     import java.io.IOException;
3     import java.nio.charset.StandardCharsets;
4     import java.nio.file.AtomicMoveNotSupportedException;
5     import java.nio.file.Files;
6     import java.nio.file.Path;
7     import java.nio.file.StandardCopyOption;
8     import java.util.Comparator;
9     import java.util.List;
10    import java.util.stream.Stream;
11
12    public class Main {
13        public static void main(String[] args) {
14            Path dir = null;
15            boolean ok = true;
16
17            try {
18                dir = Files.createTempDirectory("atomic-replace-");
19                Path target = dir.resolve("courses.txt");
20                Files.writeString(target, "старый каталог\n",
      StandardCharsets.UTF_8);
21
22                replaceCatalog(target, List.of("Java. Базовый курс",
      "Базы данных"));
23
```

```
24              String result = Files.readString(target,
    StandardCharsets.UTF_8);
25              System.out.println("Содержимое после замены:");
26              System.out.print(result);
27              String normalized = result.replace("\r\n",
    "\n").replace('\r', '\n');
28              if (!normalized.equals("Java. Базовый курс\nБазы
    данных\n")) {
29                  System.out.println("ОШИБКА: содержимое не
    совпадает");
30                  ok = false;
31              }
32          } catch (IOException e) {
33              System.err.println("Ошибка замены каталога: " +
    e.getMessage());
34              ok = false;
35          } finally {
36              deleteRecursively(dir);
37          }
38
39          if (!ok) {
40              System.exit(1);
41          }
42      }
43
44      private static void replaceCatalog(Path target, List<String>
    lines) throws IOException {
45          Path directory = target.toAbsolutePath().getParent();
46          Path temp = Files.createTempFile(directory,
    target.getFileName() + ".", ".tmp");
47          boolean moved = false;
48
49          try {
50              try (BufferedWriter writer =
    Files.newBufferedWriter(temp, StandardCharsets.UTF_8)) {
51                  for (String line : lines) {
52                      writer.write(line);
53                      writer.newLine();
54                  }
55                  writer.flush();
56              }
57
58              moveReplacing(temp, target);
59              moved = true;
60          } finally {
61              if (!moved) {
62                  Files.deleteIfExists(temp);
63              }
```

```
64          }
65      }
66
67      private static void moveReplacing(Path temp, Path target) throws
    IOException {
68          try {
69              Files.move(temp, target,
    StandardCopyOption.REPLACE_EXISTING,
    StandardCopyOption.ATOMIC_MOVE);
70              System.out.println("Замена выполнена atomic move.");
71          } catch (AtomicMoveNotSupportedException e) {
72              Files.move(temp, target,
    StandardCopyOption.REPLACE_EXISTING);
73              System.out.println("Atomic move не поддерживается:
    выполнена обычная замена, атомарность не гарантирована.");
74          }
75      }
76
77      private static void deleteRecursively(Path dir) {
78          if (dir == null) {
79              return;
80          }
81          try (Stream<Path> paths = Files.walk(dir)) {
82              paths.sorted(Comparator.reverseOrder()).forEach(path ->
    {
83                  try {
84                      Files.deleteIfExists(path);
85                  } catch (IOException ignored) {
86                  }
87              });
88          } catch (IOException ignored) {
89          }
90      }
91  }
```

Листинг – 16 7/Main.java

```
1   import java.io.BufferedReader;
2   import java.io.IOException;
3   import java.nio.charset.StandardCharsets;
4   import java.nio.file.Files;
5   import java.nio.file.Path;
6   import java.util.ArrayList;
7   import java.util.Comparator;
8   import java.util.List;
9   import java.util.stream.Stream;
10
11  public class Main {
```

```
12        public static void main(String[] args) {
13            Path dir = null;
14            boolean ok = true;
15
16            try {
17                dir = Files.createTempDirectory("import-");
18                Path bad = dir.resolve("bad.csv");
19                Path good = dir.resolve("good.csv");
20
21                Files.writeString(bad,
    "Java;72\nАлгоритмы;сто\n;10\nБазы данных;64\n",
    StandardCharsets.UTF_8);
22                Files.writeString(good, "Java;72\nБазы данных;64\n",
    StandardCharsets.UTF_8);
23
24                CatalogStore store = new CatalogStore(List.of(new
    Course("Старый курс", 10)));
25                System.out.println("До импорта: " + store.catalog());
26
27                ImportResult badResult = importCatalog(bad);
28                switch (badResult) {
29                    case ImportSuccess success -> {
30                        store.publish(success.courses());
31                        System.out.println("ОШИБКА: повреждённый импорт
    опубликован");
32                        ok = false;
33                    }
34                    case ImportFailure failure -> {
35                        System.out.println("Импорт отклонён, ошибок: " +
    failure.errors().size());
36                        for (ImportError error : failure.errors()) {
37                            System.out.println("  строка " +
    error.lineNumber() + ": " + error.message());
38                        }
39                    }
40                }
41
42                List<Course> expectedOld = List.of(new Course("Старый
    курс", 10));
43                System.out.println("После отклонённого импорта: " +
    store.catalog());
44                if (!store.catalog().equals(expectedOld)) {
45                    System.out.println("ОШИБКА: опубликована часть
    каталога");
46                    ok = false;
47                }
48
49                ImportResult goodResult = importCatalog(good);
```

```
50                switch (goodResult) {
51                    case ImportSuccess success -> {
52                        store.publish(success.courses());
53                        System.out.println("Успешный импорт опубликован:
    " + store.catalog());
54                    }
55                    case ImportFailure failure -> {
56                        System.out.println("ОШИБКА: корректный файл
    отклонён: " + failure.errors());
57                        ok = false;
58                    }
59                }
60            } catch (IOException e) {
61                System.err.println("Ошибка импорта: " + e.getMessage());
62                ok = false;
63            } finally {
64                deleteRecursively(dir);
65            }
66
67            if (!ok) {
68                System.exit(1);
69            }
70        }
71
72        private static ImportResult importCatalog(Path path) throws
    IOException {
73            List<Course> parsed = new ArrayList<>();
74            List<ImportError> errors = new ArrayList<>();
75
76            try (BufferedReader reader = Files.newBufferedReader(path,
    StandardCharsets.UTF_8)) {
77                String line;
78                int lineNumber = 0;
79
80                while ((line = reader.readLine()) != null) {
81                    lineNumber++;
82                    if (line.isBlank()) {
83                        continue;
84                    }
85
86                    String[] parts = line.split(";", -1);
87                    if (parts.length != 2) {
88                        errors.add(new ImportError(lineNumber,
    "ожидается формат название;часы"));
89                        continue;
90                    }
91
```

```
92                  String name = parts[0].trim();
93                  String hoursText = parts[1].trim();
94
95                  if (name.isEmpty()) {
96                      errors.add(new ImportError(lineNumber, "пустое
    название"));
97                      continue;
98                  }
99
100                 int hours;
101                 try {
102                     hours = Integer.parseInt(hoursText);
103                 } catch (NumberFormatException e) {
104                     errors.add(new ImportError(lineNumber, "неверное
    число часов: " + hoursText));
105                     continue;
106                 }
107
108                 if (hours <= 0) {
109                     errors.add(new ImportError(lineNumber, "часы
    должны быть положительными"));
110                     continue;
111                 }
112
113                 parsed.add(new Course(name, hours));
114             }
115         }
116
117         if (!errors.isEmpty()) {
118             return new ImportFailure(List.copyOf(errors));
119         }
120         return new ImportSuccess(List.copyOf(parsed));
121     }
122
123     private static void deleteRecursively(Path dir) {
124         if (dir == null) {
125             return;
126         }
127         try (Stream<Path> paths = Files.walk(dir)) {
128             paths.sorted(Comparator.reverseOrder()).forEach(path ->
    {
129                 try {
130                     Files.deleteIfExists(path);
131                 } catch (IOException ignored) {
132                 }
133             });
```

```
134           } catch (IOException ignored) {
135           }
136       }
137
138       private sealed interface ImportResult permits ImportSuccess,
      ImportFailure {
139       }
140
141       private record ImportSuccess(List<Course> courses) implements
      ImportResult {
142       }
143
144       private record ImportFailure(List<ImportError> errors)
      implements ImportResult {
145       }
146
147       private record ImportError(int lineNumber, String message) {
148       }
149
150       private record Course(String name, int hours) {
151       }
152
153       private static final class CatalogStore {
154           private List<Course> catalog;
155
156           private CatalogStore(List<Course> initial) {
157               this.catalog = List.copyOf(initial);
158           }
159
160           private void publish(List<Course> newCatalog) {
161               this.catalog = List.copyOf(newCatalog);
162           }
163
164           private List<Course> catalog() {
165               return catalog;
166           }
167       }
168   }
```

Листинг – 17 8/Main.java

```
1     import java.io.BufferedReader;
2     import java.io.BufferedWriter;
3     import java.io.IOException;
4     import java.nio.charset.StandardCharsets;
5     import java.nio.file.Files;
```

```
6     import java.nio.file.Path;
7     import java.util.ArrayList;
8     import java.util.List;
9
10    public class Main {
11        private static final int MAX_STORED_ERRORS = 5;
12        private static final int CANCEL_AFTER_ERRORS = 10;
13
14        public static void main(String[] args) {
15            Path smallFile = null;
16            Path largeFile = null;
17            boolean ok = true;
18
19            try {
20                Path dir = Files.createTempDirectory("stream-import-");
21                smallFile = dir.resolve("small.csv");
22                largeFile = dir.resolve("large.csv");
23
24                writeSample(smallFile, 5_000, 1_000);
25                writeSample(largeFile, 50_000, 1_000);
26
27                System.out.println("--- Небольшой файл: ошибок меньше
      порога ---");
28                ImportSummary completed = importCatalog(smallFile,
      MAX_STORED_ERRORS, CANCEL_AFTER_ERRORS);
29                printSummary(completed);
30                if (completed.cancelled()) {
31                    System.out.println("ОШИБКА: импорт не должен был
      отменяться");
32                    ok = false;
33                }
34                if (completed.totalErrors() != 5 ||
      completed.sampleErrors().size() != 5) {
35                    System.out.println("ОШИБКА: неверные счётчики
      ошибок");
36                    ok = false;
37                }
38
39                System.out.println("--- Большой файл: отмена по порогу -
      --");
40                ImportSummary cancelled = importCatalog(largeFile,
      MAX_STORED_ERRORS, CANCEL_AFTER_ERRORS);
41                printSummary(cancelled);
42                if (!cancelled.cancelled()) {
43                    System.out.println("ОШИБКА: ожидалась отмена после
      порога");
44                    ok = false;
```

```
45              }
46              if (cancelled.totalErrors() != CANCEL_AFTER_ERRORS) {
47                  System.out.println("ОШИБКА: порог отмены не
    совпал");
48                  ok = false;
49              }
50              if (cancelled.sampleErrors().size() !=
    MAX_STORED_ERRORS) {
51                  System.out.println("ОШИБКА: список ошибок не
    ограничен");
52                  ok = false;
53              }
54
55              System.out.println("Память: файл читается построчно,
    хранится не более "
56                      + MAX_STORED_ERRORS + " ошибок, валидные записи
    не накапливаются.");
57          } catch (IOException e) {
58              System.err.println("Ошибка потокового импорта: " +
    e.getMessage());
59              ok = false;
60          } finally {
61              deleteQuietly(smallFile);
62              deleteQuietly(largeFile);
63              if (smallFile != null) {
64                  deleteQuietly(smallFile.getParent());
65              }
66          }
67
68          if (!ok) {
69              System.exit(1);
70          }
71      }
72
73      private static ImportSummary importCatalog(Path path, int
    maxStoredErrors, int cancelAfterErrors)
74              throws IOException {
75          if (maxStoredErrors < 0) {
76              throw new IllegalArgumentException("maxStoredErrors не
    может быть отрицательным");
77          }
78          if (cancelAfterErrors <= 0) {
79              throw new IllegalArgumentException("cancelAfterErrors
    должен быть положительным");
80          }
81
82          long totalLines = 0;
83          long validRecords = 0;
```

```
84          BoundedErrorCollector collector = new
    BoundedErrorCollector(maxStoredErrors);
85          boolean cancelled = false;
86
87          try (BufferedReader reader = Files.newBufferedReader(path,
    StandardCharsets.UTF_8)) {
88              String line;
89              while ((line = reader.readLine()) != null) {
90                  totalLines++;
91
92                  String[] parts = line.split(";", -1);
93                  if (parts.length != 2) {
94                      collector.add(new ImportError(totalLines,
    "ожидается 2 поля"));
95                  } else {
96                      String name = parts[0].trim();
97                      String hoursText = parts[1].trim();
98
99                      if (name.isEmpty()) {
100                         collector.add(new ImportError(totalLines,
    "пустое название"));
101                     } else {
102                         try {
103                             int hours = Integer.parseInt(hoursText);
104                             if (hours <= 0) {
105                                 collector.add(new
    ImportError(totalLines, "часы должны быть положительными"));
106                             } else {
107                                 validRecords++;
108                             }
109                         } catch (NumberFormatException e) {
110                             collector.add(new
    ImportError(totalLines, "неверное число часов: " + hoursText));
111                         }
112                     }
113                 }
114
115                 if (collector.totalErrors() >= cancelAfterErrors) {
116                     cancelled = true;
117                     break;
118                 }
119             }
120         }
121
122         return new ImportSummary(totalLines, validRecords,
    collector.totalErrors(),
123                 collector.storedErrors(), cancelled);
```

```
124      }
125
126      private static void writeSample(Path path, int totalLines, int
     errorEvery) throws IOException {
127          try (BufferedWriter writer = Files.newBufferedWriter(path,
     StandardCharsets.UTF_8)) {
128              for (int i = 1; i <= totalLines; i++) {
129                  if (i % errorEvery == 0) {
130                      writer.write("Курс " + i + ";ошибка");
131                  } else {
132                      writer.write("Курс " + i + ";" + (36 + (i % 5) *
     12));
133                  }
134                  writer.newLine();
135              }
136          }
137      }
138
139      private static void printSummary(ImportSummary summary) {
140          System.out.println("Обработано строк: " +
     summary.totalLines());
141          System.out.println("Корректных записей: " +
     summary.validRecords());
142          System.out.println("Всего ошибок: " +
     summary.totalErrors());
143          System.out.println("Сохранено ошибок: " +
     summary.sampleErrors().size());
144          for (ImportError error : summary.sampleErrors()) {
145              System.out.println("  строка " + error.lineNumber() + ":
     " + error.message());
146          }
147          System.out.println("Импорт отменён: " +
     summary.cancelled());
148      }
149
150      private static void deleteQuietly(Path path) {
151          if (path == null) {
152              return;
153          }
154          try {
155              Files.deleteIfExists(path);
156          } catch (IOException ignored) {
157          }
158      }
159
160      private record ImportSummary(long totalLines, long validRecords,
     long totalErrors,
```

```
161                                     List<ImportError> sampleErrors,
    boolean cancelled) {
162     }
163
164     private record ImportError(long lineNumber, String message) {
165     }
166
167     private static final class BoundedErrorCollector {
168         private final int maxStored;
169         private final List<ImportError> stored = new ArrayList<>();
170         private long total;
171
172         private BoundedErrorCollector(int maxStored) {
173             this.maxStored = maxStored;
174         }
175
176         private void add(ImportError error) {
177             total++;
178             if (stored.size() < maxStored) {
179                 stored.add(error);
180             }
181         }
182
183         private long totalErrors() {
184             return total;
185         }
186
187         private List<ImportError> storedErrors() {
188             return List.copyOf(stored);
189         }
190     }
191 }
```

Листинг – 18 9/Main.java

```
1   import java.io.BufferedInputStream;
2   import java.io.BufferedOutputStream;
3   import java.io.ByteArrayInputStream;
4   import java.io.ByteArrayOutputStream;
5   import java.io.DataInputStream;
6   import java.io.DataOutputStream;
7   import java.io.EOFException;
8   import java.io.IOException;
9   import java.io.UTFDataFormatException;
10  import java.nio.file.Files;
11  import java.nio.file.Path;
```

```
12  import java.util.ArrayList;
13  import java.util.Comparator;
14  import java.util.List;
15  import java.util.stream.Stream;
16  import java.util.zip.CRC32;
17
18  public class Main {
19      private static final int MAGIC = 0x4341544C;
20      private static final int VERSION_1 = 1;
21      private static final int VERSION_2 = 2;
22      private static final int MAX_PAYLOAD = 64 * 1024 * 1024;
23      private static final int MAX_RECORDS = 100_000;
24
25      public static void main(String[] args) {
26          Path dir = null;
27          boolean ok = true;
28
29          try {
30              dir = Files.createTempDirectory("versioned-format-");
31              List<Course> source = List.of(
32                      new Course("Java. Базовый курс", 72, 2),
33                      new Course("Базы данных", 108, 3)
34              );
35
36              Path v2 = dir.resolve("catalog-v2.bin");
37              writeCatalog(v2, VERSION_2, source);
38
39              Publisher publisher = new Publisher(List.of(new
    Course("Старый курс", 36, 1)));
40              System.out.println("До публикации: " +
    publisher.catalog());
41
42              LoadedCatalog loadedV2 = readCatalog(v2);
43              publisher.publish(loadedV2.courses());
44              System.out.println("v2 прочитан (версия " +
    loadedV2.sourceVersion() + "): " + publisher.catalog());
45
46              Path v1 = dir.resolve("catalog-v1.bin");
47              writeCatalog(v1, VERSION_1, source);
48              LoadedCatalog loadedV1 = readCatalog(v1);
49              System.out.println("v1 миграция -> v2 (версия " +
    loadedV1.sourceVersion() + "): " + loadedV1.courses());
50
51              Path future = dir.resolve("catalog-v3.bin");
52              writeRawVersion(future, 3);
53              ok &= expectRejected(future, "будущая версия");
```

```
54
55              Path corrupted = dir.resolve("catalog-corrupted.bin");
56              byte[] bytes = Files.readAllBytes(v2);
57              bytes[bytes.length - 1] ^= 0x5A;
58              Files.write(corrupted, bytes);
59
60              List<Course> before = publisher.catalog();
61              ok &= expectRejected(corrupted, "повреждённый файл");
62              if (!publisher.catalog().equals(before)) {
63                  System.out.println("ОШИБКА: повреждённый файл
    изменил опубликованный каталог");
64                  ok = false;
65              } else {
66                  System.out.println("Опубликованный каталог не
    изменён после повреждения.");
67              }
68          } catch (IOException e) {
69              System.err.println("Ошибка версионируемого формата: " +
    e.getMessage());
70              ok = false;
71          } finally {
72              deleteRecursively(dir);
73          }
74
75          if (!ok) {
76              System.exit(1);
77          }
78      }
79
80      private static void writeCatalog(Path path, int version,
    List<Course> courses) throws IOException {
81          if (version != VERSION_1 && version != VERSION_2) {
82              throw new IllegalArgumentException("неподдерживаемая
    версия для записи: " + version);
83          }
84
85          ByteArrayOutputStream payloadBytes = new
    ByteArrayOutputStream();
86          try (DataOutputStream payload = new
    DataOutputStream(payloadBytes)) {
87              payload.writeInt(courses.size());
88              for (Course course : courses) {
89                  payload.writeUTF(course.name());
90                  payload.writeInt(course.hours());
91                  if (version == VERSION_2) {
92                      payload.writeInt(course.credits());
93                  }
```

```
94              }
95          }
96
97          byte[] payload = payloadBytes.toByteArray();
98          CRC32 crc = new CRC32();
99          crc.update(payload);
100
101         try (DataOutputStream out = new DataOutputStream(
102                 new
    BufferedOutputStream(Files.newOutputStream(path)))) {
103             out.writeInt(MAGIC);
104             out.writeInt(version);
105             out.writeInt(payload.length);
106             out.writeInt((int) crc.getValue());
107             out.write(payload);
108         }
109     }
110
111     private static LoadedCatalog readCatalog(Path path) throws
    IOException {
112         try (DataInputStream in = new DataInputStream(
113                 new
    BufferedInputStream(Files.newInputStream(path)))) {
114
115             int magic;
116             int version;
117             int payloadLength;
118             int expectedChecksum;
119
120             try {
121                 magic = in.readInt();
122                 if (magic != MAGIC) {
123                     throw new CatalogFormatException("неверный magic
    header");
124                 }
125
126                 version = in.readInt();
127                 if (version != VERSION_1 && version != VERSION_2) {
128                     throw new UnsupportedVersionException(version);
129                 }
130
131                 payloadLength = in.readInt();
132                 if (payloadLength < 0 || payloadLength >
    MAX_PAYLOAD) {
133                     throw new CatalogFormatException("недопустимая
    длина payload: " + payloadLength);
```

```
134                 }
135
136                 expectedChecksum = in.readInt();
137             } catch (EOFException e) {
138                 throw new CatalogFormatException("заголовок файла
    обрезан", e);
139             }
140
141             byte[] payload = in.readNBytes(payloadLength);
142             if (payload.length != payloadLength) {
143                 throw new CatalogFormatException("payload обрезан:
    ожидалось "
144                         + payloadLength + " байт, получено " +
    payload.length);
145             }
146
147             if (in.read() != -1) {
148                 throw new CatalogFormatException("лишние данные
    после payload");
149             }
150
151             CRC32 crc = new CRC32();
152             crc.update(payload);
153             if ((int) crc.getValue() != expectedChecksum) {
154                 throw new CatalogFormatException("checksum не
    совпал: данные повреждены");
155             }
156
157             return parsePayload(version, payload);
158         }
159     }
160
161     private static LoadedCatalog parsePayload(int version, byte[]
    payload) throws IOException {
162         try (DataInputStream in = new DataInputStream(new
    ByteArrayInputStream(payload))) {
163             int count;
164             try {
165                 count = in.readInt();
166             } catch (EOFException e) {
167                 throw new CatalogFormatException("payload обрезан:
    нет количества записей", e);
168             }
169
170             if (count < 0 || count > MAX_RECORDS) {
171                 throw new CatalogFormatException("недопустимое
    количество записей: " + count);
```

```
172             }
173
174             List<Course> courses = new ArrayList<>(count);
175             for (int i = 0; i < count; i++) {
176                 try {
177                     String name = in.readUTF();
178                     int hours = in.readInt();
179                     int credits = version == VERSION_1 ?
    creditsFromHours(hours) : in.readInt();
180                     validateCourse(name, hours, credits, i + 1);
181                     courses.add(new Course(name, hours, credits));
182                 } catch (EOFException e) {
183                     throw new CatalogFormatException("payload
    обрезан в записи #" + (i + 1), e);
184                 } catch (UTFDataFormatException e) {
185                     throw new CatalogFormatException("повреждённая
    строка в записи #" + (i + 1), e);
186                 }
187             }
188
189             if (in.available() != 0) {
190                 throw new CatalogFormatException("после записей
    остались лишние байты");
191             }
192
193             return new LoadedCatalog(version, List.copyOf(courses));
194         }
195     }
196
197     private static int creditsFromHours(int hours) {
198         return Math.max(1, hours / 36);
199     }
200
201     private static void validateCourse(String name, int hours, int
    credits, int index)
202             throws CatalogFormatException {
203         if (name == null || name.isBlank()) {
204             throw new CatalogFormatException("пустое название в
    записи #" + index);
205         }
206         if (hours <= 0) {
207             throw new CatalogFormatException("неверные часы в записи
    #" + index);
208         }
209         if (credits <= 0) {
210             throw new CatalogFormatException("неверные кредиты в
    записи #" + index);
```

```
211          }
212      }
213
214      private static void writeRawVersion(Path path, int version)
     throws IOException {
215          byte[] payload = new byte[] {0, 0, 0, 0};
216          CRC32 crc = new CRC32();
217          crc.update(payload);
218
219          try (DataOutputStream out = new DataOutputStream(
220                  new
     BufferedOutputStream(Files.newOutputStream(path)))) {
221              out.writeInt(MAGIC);
222              out.writeInt(version);
223              out.writeInt(payload.length);
224              out.writeInt((int) crc.getValue());
225              out.write(payload);
226          }
227      }
228
229      private static boolean expectRejected(Path path, String label) {
230          try {
231              LoadedCatalog catalog = readCatalog(path);
232              System.out.println("ОШИБКА: " + label + " принят: " +
     catalog);
233              return false;
234          } catch (UnsupportedVersionException e) {
235              System.out.println(label + " отклонён: " +
     e.getMessage());
236              return true;
237          } catch (CatalogFormatException e) {
238              System.out.println(label + " отклонён: " +
     e.getMessage());
239              return true;
240          } catch (IOException e) {
241              System.out.println(label + " отклонён (I/O): " +
     e.getMessage());
242              return true;
243          }
244      }
245
246      private static void deleteRecursively(Path dir) {
247          if (dir == null) {
248              return;
249          }
250          try (Stream<Path> paths = Files.walk(dir)) {
```

```
251             paths.sorted(Comparator.reverseOrder()).forEach(path ->
    {
252                 try {
253                     Files.deleteIfExists(path);
254                 } catch (IOException ignored) {
255                 }
256             });
257         } catch (IOException ignored) {
258         }
259     }
260
261     private record Course(String name, int hours, int credits) {
262     }
263
264     private record LoadedCatalog(int sourceVersion, List<Course>
    courses) {
265     }
266
267     private static class CatalogFormatException extends IOException
    {
268         private static final long serialVersionUID = 1L;
269
270         private CatalogFormatException(String message) {
271             super(message);
272         }
273
274         private CatalogFormatException(String message, Throwable
    cause) {
275             super(message, cause);
276         }
277     }
278
279     private static final class UnsupportedVersionException extends
    CatalogFormatException {
280         private static final long serialVersionUID = 1L;
281
282         private UnsupportedVersionException(int version) {
283             super("неизвестная версия формата: " + version);
284         }
285     }
286
287     private static final class Publisher {
288         private List<Course> catalog;
289
290         private Publisher(List<Course> initial) {
291             this.catalog = List.copyOf(initial);
```

```
292           }
293
294           private void publish(List<Course> newCatalog) {
295               this.catalog = List.copyOf(newCatalog);
296           }
297
298           private List<Course> catalog() {
299               return catalog;
300           }
301       }
302   }
```